Architetture dei Sistemi di Elaborazione
02GOLYG

Computer Architectures
02LSEYG

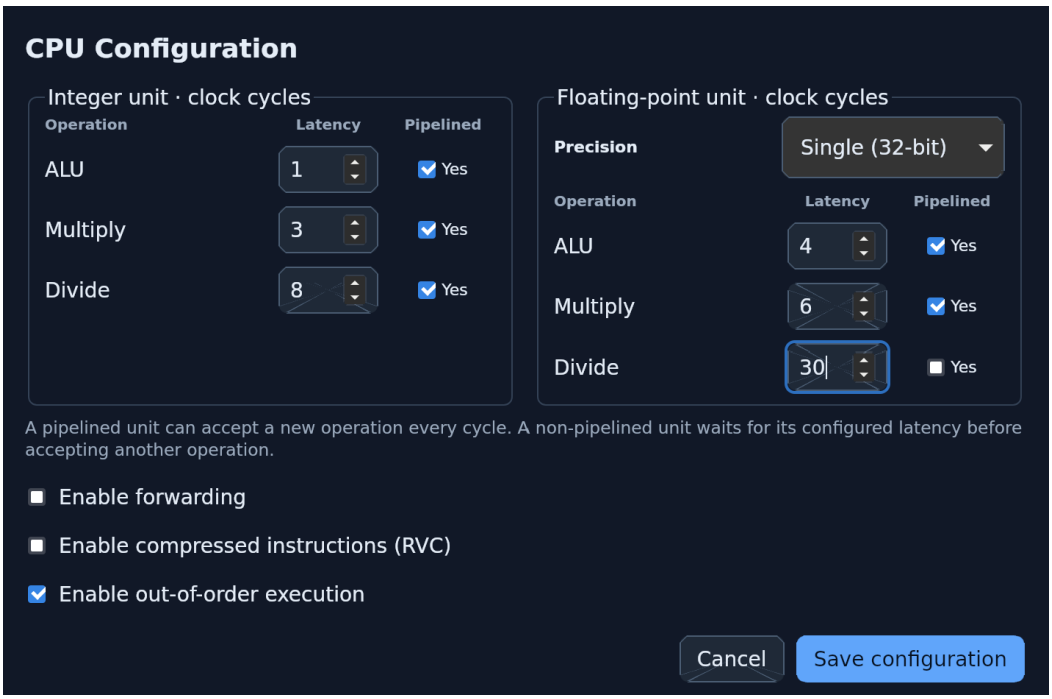
Laboratory 0x02

Expected submission:
SURNAME_NAME_MATRICOLA_lab_02.zip

Including:
- program_1
- lab_02.pdf (complete this file and export it as PDF)

Submission deadline: 13/10/2026

Please configure the **in-order CPU** with the *Initial Configuration* provided below (please note forwarding is **NOT** enabled) :

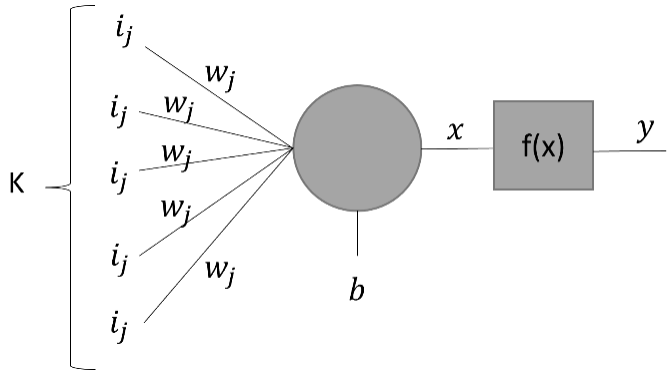


1) Write a RISC-V assembly program (**`program_1.s`**) able to compute the output (y) of a **neural computation** (see the Fig. below):

$$x = \sum_{j=0}^{K-1}(i_j * w_j) + b$$
$$y = f(x)$$

The activation function f is defined as:

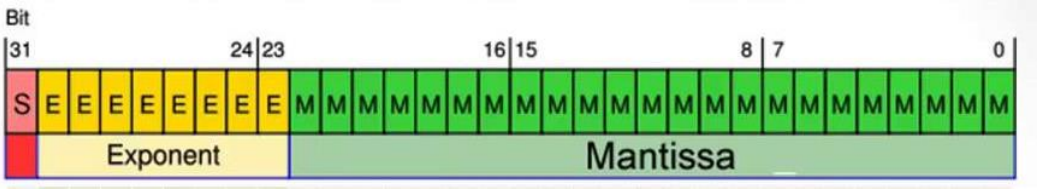
$$f(x) = \begin{cases} 0, & if\ the\ exponent\ part\ of\ x\ is\ equal\ to\ 0x7f\ or\ x == 0 \\ 1/x, & otherwise \end{cases}$$



Assume the vectors i and w respectively store the inputs entering the neuron and the weights of the connections. They contain $K=30$ **single precision floating point** elements. Assume that b is a **single precision floating point** constant and is equal to $0xab$, and y is a single precision **floating point** value stored in memory.

Compute y.

To easily identify the exponent of a floating-point number, the IEEE 754 single-precision binary floating-point format is shown below:



- **Calculate** (manually) the data memory footprint of your program:

Data	Number of bytes
Inputs	
Weights	
b	
x	
y	
Total	

- Calculate the CPU performance equation (CPU time) of the above program by assuming a clock frequency of 15 MHz:

$$\textbf{CPU time} = (\sum_{i=1}^{n} \textbf{CPI}_i \times \textbf{IC}_i) \times \textbf{Clock cycle period}$$

By definition:

- CPI is equal to the number of clock cycles required by the related functional unit to execute the instruction (EX stage).
- IC_i is the number of times an instruction is repeated in the referenced source code.

Recalculate the CPU performance equation assuming that you can triple the speed by just one unit of your choice between the FP multiplier or the FP divider:

- FP multiplier unit: 6 → 2 clock cycles
  or
- FP divider unit: 30 → 10 clock cycles

Table 1: CPU time by hand

	Initial CPU time (a)	CPU time (b – MUL speeded up)	CPU time (b – DIV speeded up)
program_1.s			

Using the simulator, calculate the CPU time again and fill in the following table:

Table 2: CPU time using the simulator

	Initial CPU time (a)	CPU time (b – MUL speeded up)	CPU time (b – DIV speeded up)
program_1.s			

Are there any differences? If so, where and why? If not, please provide some comments in the box below:

Your answer:

- From the *Initial Configuration*, enable the Forwarding option and compute how many clock cycles the program takes to execute.

Table 3: forwarding enabled

	Number of clock cycles	IPC (Instructions Per Clock)
program_1.s		

Are there any differences? If so, where and why? If not, please provide some comments in the box below:

Your answer:

2) Enable forwarding on CPU configuration, then validate experimentally the Amdahl's law, defined as follows:

$$\text{speedup}_{\text{overall}} = \frac{\text{execution time}_{\text{old}}}{\text{execution time}_{\text{new}}} = \frac{1}{(1 - \text{fraction}_{\text{enhanced}}) + \frac{\text{fraction}_{\text{enhanced}}}{\text{speedup}_{\text{enhanced}}}}$$

a. Using the program developed before: **program_1.s**

b. Modify the processor architectural parameters related to multicycle instructions in the following way:

1) Configuration 1
   - Starting from the *Initial Configuration*, change the FP addition latency to 3
2) Configuration 2
   - Starting from the *Initial Configuration*, change the FP multiplier latency to 4
3) Configuration 3
   - Starting from the *Initial Configuration*, change the FP division latency to 10

Compute both manually (using the Amdahl's Law) and with the simulator the speed-up for any one of the previous processor configurations. Compare the obtained results and complete the following table.

Table 5: **program_1.s speed-up computed by hand and by simulation**

Proc. Config. / **Speed-up comp.**	Initial config. [c.c.]	Config. 1	Config. 2	Config. 3
By hand				
By simulation				